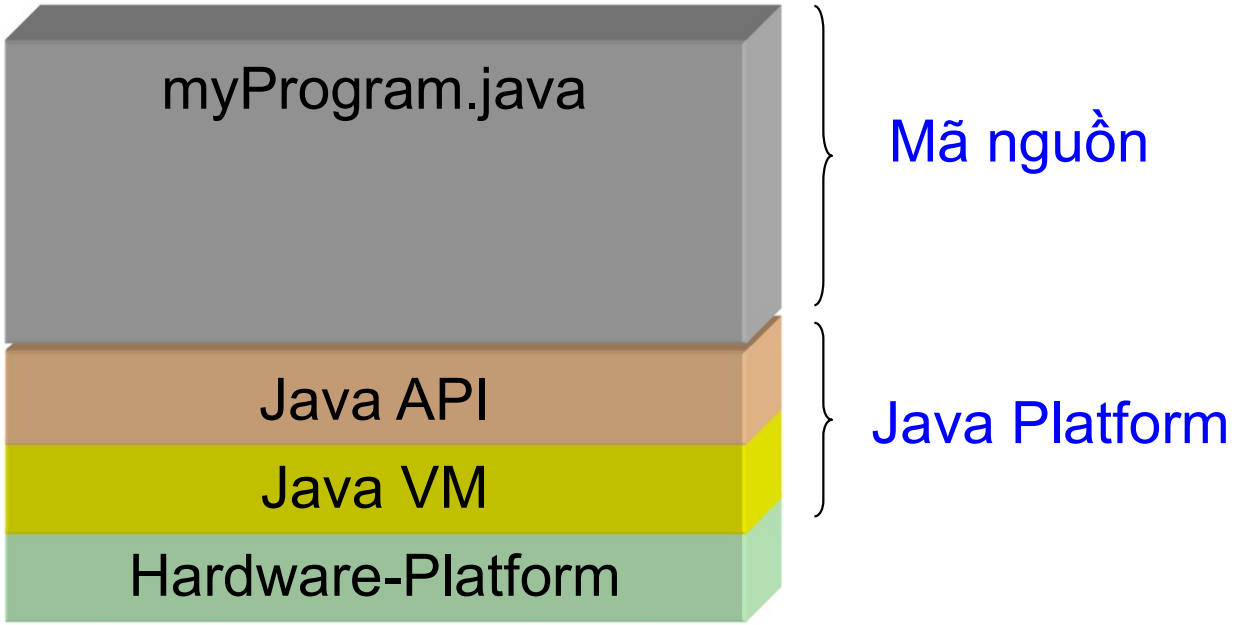
KIẾN TRÚC CỦA JAVA

- ***Java Platform***
  - Java Virtual Machine (Java VM)
  - Java Application Programming Interface (Java API)



Khoa Công Nghệ Thông Tin

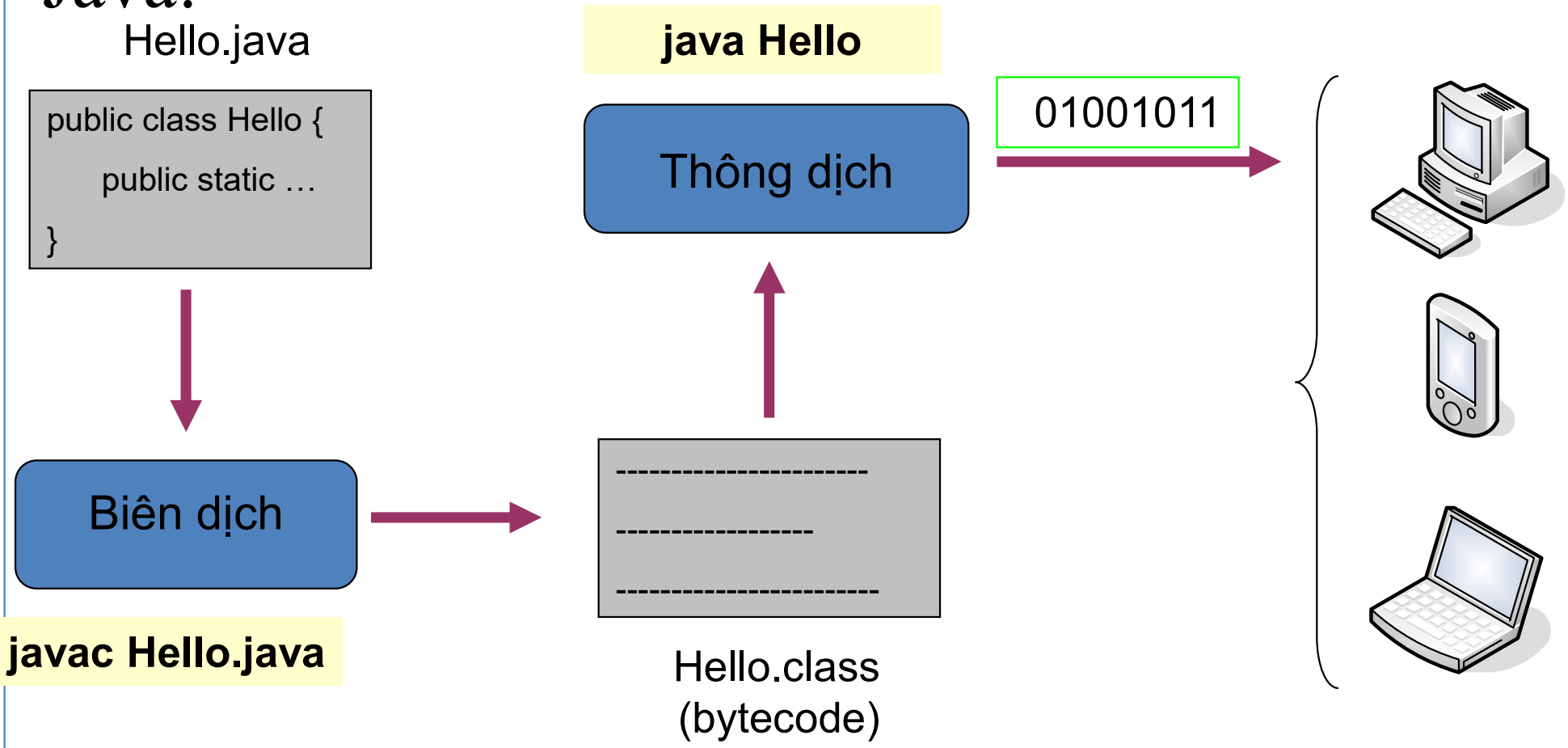
anhminhde81@gmail.com

KIẾN TRÚC CỦA JAVA

- Thư viện lớp Java: bộ JDK bao gồm rất nhiều lớp chuẩn đã được xây dựng sẵn.
- Lập trình viên thường sử dụng các lớp chuẩn để phát triển ứng dụng.
- Các gói chuẩn của Java:
  - **java.lang**
  - java.applet
  - java.awt
  - **java.io**
  - java.util
  - java.net
  - java.awt.event
  - java.rmi
  - java.security
  - java.sql

CÁC BƯỚC PHÁT TRIỂN

- Các bước phát triển một chương trình bằng Java:



anhminhde81@gmail.com

Ví dụ một chương trình Java mẫu

```
//  This is a simple program called “Ex1.java”
// import java.lang.*;
class Ex1
{
    public static void main(String args[])
    {
        System.out.println(“My first program in Java”);
    }
}
```

Java Program Structure

```
//  comments about the class
public class MyProgram
{
    //  comments about the method
    public static void main (String[] args)
    {
    }
}
```

method header

method body

anhminhde81@gmail.com

Biên dịch chương trình java

- **..\jdk\bin>javac Ex1.java**
- **..\jdk\bin>java Ex1**

- Kết quả:

  My first program in Java

CẤU TRÚC MỘT CHƯƠNG TRÌNH CƠ BẢN

```
1   // Tên file : Hello.java
2   /* Tác giả : Barak Obama*/
3
4    public class Hello
5    {
6     // Phương thức main, điểm bắt đầu của chương trình
7     public static void main( String args[ ] )
8     {
9        System.out.println( "Hello World" );
10
11    } // Kết thúc phương thức main
12
13  }//
```

Tên lớp chứa hàm main phải giống tên file

Điểm bắt đầu và kết thúc của lớp

Dấu hiệu chú thích =>
Làm cho chương trình dễ hiểu hơn. Trình biên dịch sẽ bỏ qua những dòng có dấu chú thích

Khai báo lớp

Mỗi CT phải có ít nhất một khai báo lớp

Phương thức main() sẽ được gọi đầu tiên. Mỗi CT thực thi phải có một phương thức main()

Hiển thị dãy ký tự ra màn hình

Các câu lệnh phải kết thúc bằng dấu chấm phẩy

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

GV: Võ Tấn Dũng

PHƯƠNG THỨC MAIN

- **Phương thức main():** là điểm bắt đầu thực thi một ứng dụng.
- Mỗi ứng dụng Java phải chứa một phương thức main có dạng như sau: **public static void main(String[] args)**
- Phương thức main chứa ba bổ từ đặc tả sau:

- public: chỉ ra rằng phương thức main có thể được gọi bởi bất kỳ đối tượng nào.

- static: chỉ ra rằng phương thức main là một phương thức lớp.

- void: chỉ ra rằng phương thức main sẽ không trả về bất kỳ một giá trị nào.

BIÊN DỊCH VÀ THỰC THI

- Biên dịch chương trình
  - Vào chế độ Console của Windows
  - Gõ câu lệnh ***javac Hello.java***
  - Nếu không có thông báo lỗi, file *Hello.class* sẽ được tạo ra
- Thực thi chương trình
  - Gõ câu lệnh ***java Hello*** *(không cần .class)*



anhminhde81@gmail.com

GV: Võ Tấn Dũng

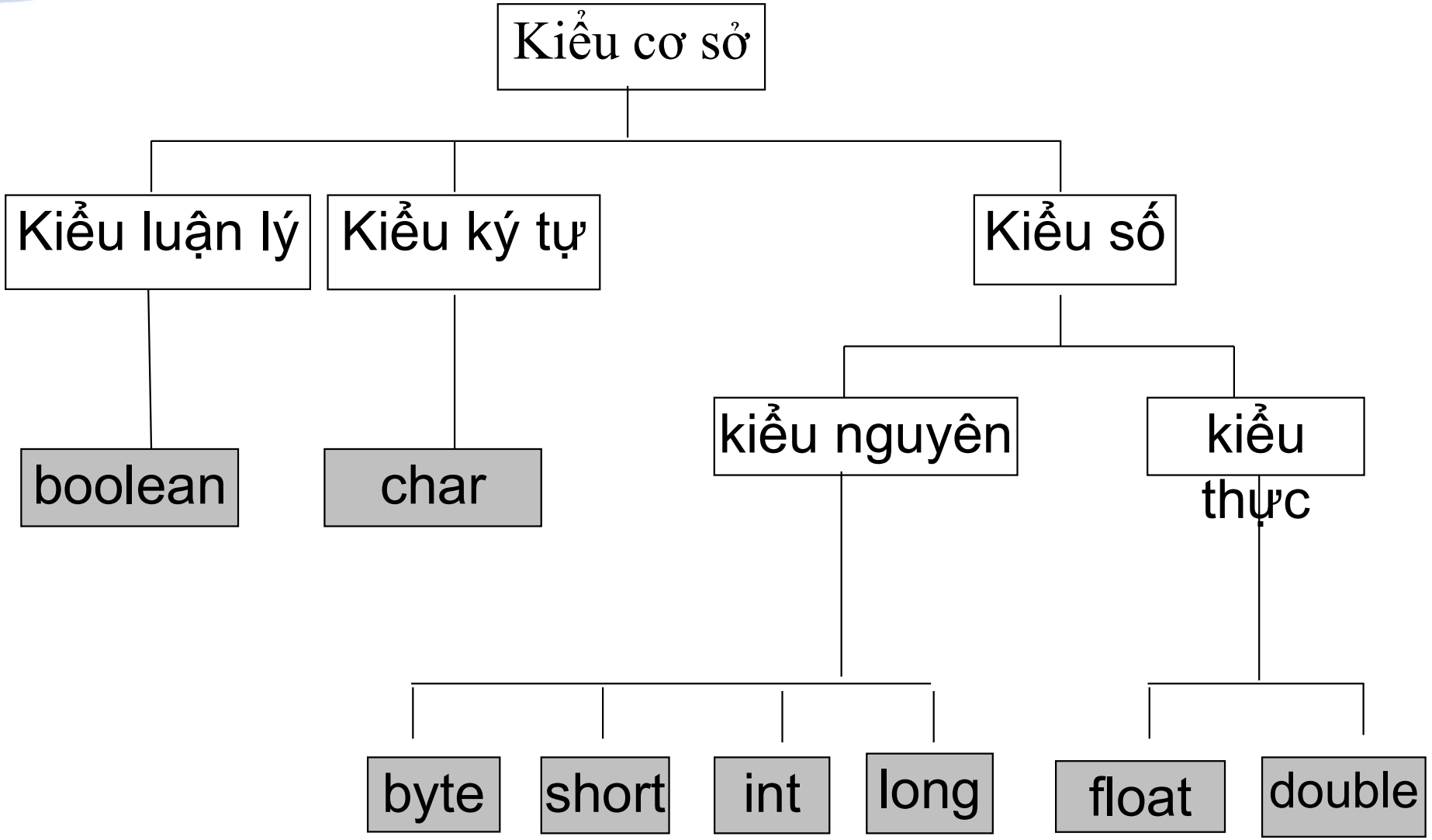
Kiểu dữ liệu

- Kiểu dữ liệu cơ sở (Primitive Data Types)
- Kiểu dữ liệu tham chiếu (Reference data types)

Kiểu dữ liệu cơ sở

- byte
- char
- boolean
- short
- int
- long
- float
- double

KIỂU DỮ LIỆU CƠ SỞ (primitive type)

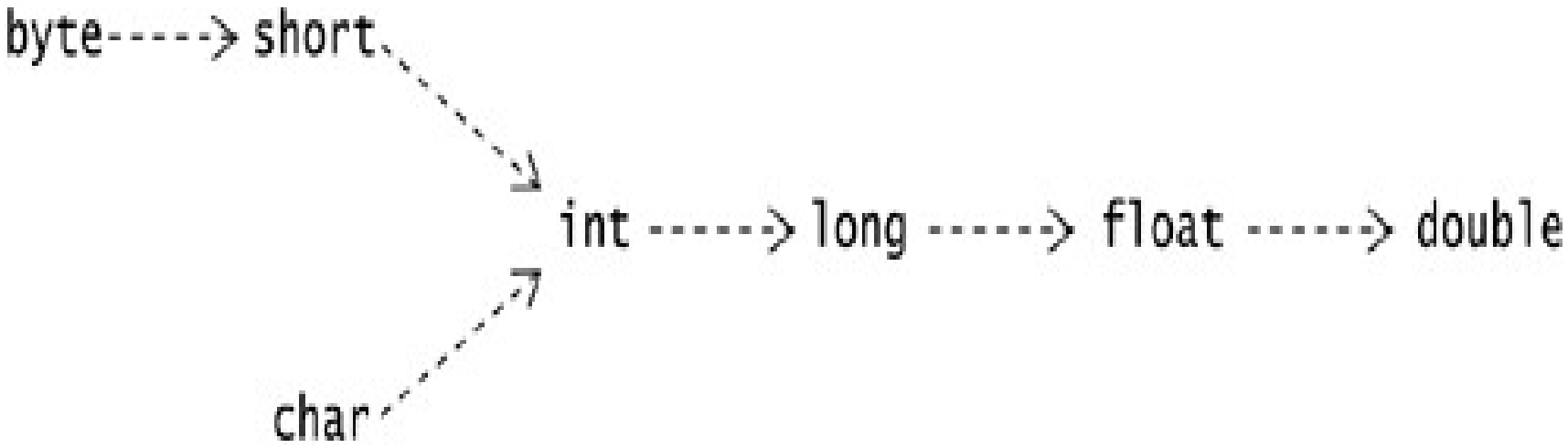


anhminhde81@gmail.com

GV: Võ Tấn Dũng

KIỂU DỮ LIỆU CƠ SỞ

- Khi gặp phải sự không tương thích kiểu dữ liệu chúng ta phải tiến hành chuyển đổi kiểu dữ liệu cho biến hoặc biểu thức



anhminhde81@gmail.com

GV: Võ Tấn Dũng

KIỂU DỮ LIỆU CƠ SỞ

- Kiểu số nguyên

Kiểu	Kích thước	Khoảng giá trị
byte	8 bits	-256…255
short	16 bits	-32768…32767
int	32 bits	$-2^{32}…2^{32}-1$
long	64 bits	$-2^{64}…2^{64}-1$

- Kiểu số thực

Kiểu	Kích thước	Khoảng giá trị
float	32 bits	-3.4e38…3.4e38
double	64 bits	-1.7e308…1.7e308

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

GV: Võ Tấn Dũng

KIỂU DỮ LIỆU CƠ SỞ

- Kiểu **boolean**: Nhận giá trị true hoặc false
- Kiểu **char**: Kiểu ký tự theo chuẩn Unicode

  Một số hằng ký tự:

Ký tự	Ý nghĩa
\b	Xóa lùi (BackSpace)
\t	Tab
\n	Xuống hàng
\r	Dấu enter
\”	Nháy kép
\’	Nháy đơn
\\	Số ngược
\f	Đẩy trang
\uxxxx	Ký tự unicode

GV: Võ Tấn Dũng

Kiểu dữ liệu cơ sở

- 4 kiểu integers:
  - `byte`, `short`, `int`, `long`
- 2 kiểu số thực:
  - `float`, `double`
- 1 kiểu characters:
  - `char`
- 1 kiểu boolean :
  - `boolean`

Kiểu dữ liệu cơ sở

Type	Storage	Min Value	Max Value
byte	8 bits	-128	127
short	16 bits	-32,768	32,767
int	32 bits	-2,147,483,648	2,147,483,647
long	64 bits	$< -9 \times 10^{18}$	$> 9 \times 10^{18}$
float	32 bits	$+/- 3.4 \times 10^{38}$ with 7 significant digits	
double	64 bits	$+/- 1.7 \times 10^{308}$ with 15 significant digits	

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

Characters

- Character literals are delimited by single quotes:

```
'a'    'X'    '7'    '$'    ','    '\n'
```

- The ASCII characters :

uppercase letters	A, B, C, …
lowercase letters	a, b, c, …
Punctuation	period, semi-colon, …
digits	0, 1, 2, …
special symbols	&, |, \, …
control characters	carriage return, tab, ...

anhminhde81@gmail.com

Boolean

- A `boolean` value represents a true or false condition

- A boolean can also be used to represent any two states, such as a light bulb being on or off

- The reserved words `true` and `false` are the only valid values for a boolean type

```
boolean done = false;
```


anhminhde81@gmail.com

Kiểu dữ liệu tham chiếu

- Mảng (Array)
- Lớp (Class)
- Interface

Ép kiểu (Type Casting)

- Kiểu dữ liệu này được chuyển đổi sang một kiểu dữ liệu khác
- Ví dụ

```
float c = 34.89675f;
int b = (int)c + 10;
```

KIỂU DỮ LIỆU CƠ SỞ

- **Toán tử ép kiểu:**

  <tên biến> = (kiểu_dữ_liệu) <tên_biến>;

  float fNum = 2.2;

  int iCount = (int) fNum

- **Ép kiểu rộng** (widening conversion): từ kiểu nhỏ sang kiểu lớn (không mất mát thông tin)
- **Ép kiểu hẹp** (narrow conversion): từ kiểu lớn sang kiểu nhỏ (có khả năng mất mát thông tin)

Biến số

- Cú pháp

  datatype identifier [=value][, identifier[=value]...];

- Khai báo biến số gồm 3 thành phần:
  - Kiểu dữ liệu của biến số
  - Tên biến
  - Giá trị ban đầu của biến (không bắt buộc)

Bộ từ khóa của Java

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	

anhminhde81@gmail.com

TỪ KHÓA (keyword)

- Từ khóa cho các kiểu dữ liệu cơ bản : **byte, short, int, long, float, double, char, boolean.**
- Từ khóa cho phát biểu lặp: **do, while, for, break, continue.**
- Từ khóa cho phát biểu rẽ nhánh: **if, else, switch, case, default, break.**
- Từ khóa đặc tả đặc tính một method: **private, public, protected, final, static, abstract, synchronized.**
- Hằng (literal): **true, false, null.**
- Từ khóa liên quan đến method: **return, void.**
- Từ khoá liên quan đến package: **package, import.**
- Từ khóa cho việc quản lý lỗi: **try, catch, finally, throw, throws.**
- Từ khóa liên quan đến đối tượng: **new, extends, implements, class, instanceof, this, super.**

anhminhde81@gmail.com

GV: Võ Tấn Dũng

Khai báo mảng

- Ba cách để khai báo mảng:
  - **datatype identifier [ ];**
  - **datatype identifier [ ] = new datatype[size];**
  - **datatype identifier [ ]= {value1,value2,….valueN};**

MẢNG

Mảng là tập hợp nhiều phần tử có cùng tên, cùng kiểu dữ liệu và mỗi phần tử trong mảng được truy xuất thông qua chỉ số của nó trong mảng.

Khai báo mảng:

<kiểu dữ liệu> <tên mảng>[];

<kiểu dữ liệu>[] <tên mảng>;

***Ví dụ*:**

```
int arrInt[];
int[] arrInt;
int[] arrInt1, arrInt2, arrInt3;
```


anhminhde81@gmail.com

GV: Võ Tấn Dũng

MẢNG

Khởi tạo mảng:

Chúng ta có thể khởi tạo giá trị ban đầu cho các phần tử của mảng khi nó được khai báo.

Ví dụ:

int arrInt[] = {1, 2, 3};

char arrChar[] = {'a', 'b', 'c'};

String arrString[] = {"ABC", "EFG", 'GHI'};

MẢNG

Truy cập mảng:

- Chỉ số mảng trong Java bắt đầu tư 0. Vì vậy phần tử đầu tiên có chỉ số là 0, và phần tử thứ n có chỉ số là n-1. Các phần tử của mảng được truy xuất thông qua chỉ số của nó được đặt giữa cặp dấu ngoặc vuông [].

- ***Ví dụ***:

```
int arrInt[] = {1, 2, 3};
int x = arrInt[0]; // x sẽ có giá trị là 1.
int y = arrInt[1]; // y sẽ có giá trị là 2.
int z = arrInt[2]; // z sẽ có giá trị là 3.
```


anhminhde81@gmail.com

GV: Võ Tấn Dũng

MẢNG

- Nhập và xuất giá trị các phần tử của một mảng các số nguyên:

```
class ArrayDemo
{
    public static void main(String args[])
    {
        int arrInt[] = new int[10];
        int i;
        for(i = 0; i < 10; i = i+1)
                arrInt[i] = i;
        for(i = 0; i < 10; i = i+1)
                System.out.println("This is arrInt[" + i +"]: " +
    arrInt[i]);
    }
}
```

MẢNG

- Tìm phần tử có giá trị nhỏ nhất (Min) và lớn nhất (Max) trong một mảng.

```
class MinMax2
{    public static void main(String args[])
     {   int nums[] = { 99, -10, 100123, 18, -978, 5623, 463, -9, 287, 49 };
         int min, max;
         min = max = nums[0];
         for(int i=1; i < 10; i++)
         {
                  if(nums[i] < min) min = nums[i];
                  if(nums[i] > max) max = nums[i];
         }
         System.out.println("Min and max: " + min + " " + max);
     }
}
```

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

MẢNG

- Nhập và xuất giá trị của các phần tử trong một mảng hai chiều.

```
class TwoD_Arr
{    public static void main(String args[])
     {   int t, i;
         int table[][] = new int[3][4];
         for(t=0; t < 3; ++t)
         {        for(i=0; i < 4; ++i)
                  {        table[t][i] = (t*4)+i+1;
                           System.out.print(table[t][i] + " ");
                  }
                  System.out.println();
         }
     }
}
```

CHUỖI TRONG JAVA (STRING)

- Trong những ngôn ngữ lập trình khác (C chẳng hạn) → một chuỗi được xem như một mảng các ký tự.
- Trong java thì khác, java cung cấp một lớp String để làm việc với đối tượng dữ liệu chuỗi cùng các thao tác trên đối tượng dữ liệu này.

CHUỖI

- Nhập ký tự từ bàn phím:

```
import java.io.*;
class InputChar
{    public static void main(String args[])
     {   char ch = '';
         try{
                  ch = (char) System.in.read();
             }
         catch(Exception e){
                  System.out.println("Nhập lỗi!");
         }
         System.out.println("Ky tu vua nhap:" + ch);
     }
}
```

- **Nhập dữ liệu số**

```
import java.io.*;
class inputNum
{    public static void main(String[] args)
     {   int n=0;
         try { BufferedReader in = new BufferedReader(
                              new InputStreamReader(System.in));
           String s;
           s = in.readLine();
           n = Integer.parseInt(s);
         }
         catch(Exception e){
           System.out.println("Nhập dữ liệu bị lỗi !");
         }
         System.out.println("Bạn vừa nhập số:" + n);
     }
```


anhminhde81@gmail.com

CHUỖI

- **Tạo đối tượng chuỗi**

```
class StringDemo
{
    public static void main(String args[])
    {
        // Tao chuoi bang nhieu cach khac nhau
        String str1 = new String("Chuoi trong java la nhung Objects.");
        String str2 = "Chung duoc xay dung bang nhieu cach khac
    nhau.";
        String str3 = new String(str2);
        System.out.println(str1);
        System.out.println(str2);
        System.out.println(str3);
    }
}
```

CHUỖI

- **Minh họa một số thao tác cơ bản trên chuỗi**

```
// Chuong trinh minh hoa cac thao tac tren chuoi ky tu
class StrOps
{    public static void main(String args[])
     {   String str1 = "Java la chon lua so mot cho lap trinh ung dung Web.";
         String str2 = new String(str1);
         String str3 = "Java ho tro doi tuong String de xu ly chuoi";
         int result, idx;
         char ch;
         System.out.println("str1:" + str1);
         System.out.println("str2:" + str2);
         System.out.println("str3:" + str3);
         System.out.println("Chieu dai cua chuoi str1 la:" + str1.length());
         // Hien thi chuoi str1, moi lan mot ky tu.
         System.out.println();
         for(int i=0; i < str1.length(); i++)
```

anhminhdo81@gmail.com

CHUỖI

```
System.out.println();
if(str1.equals(str2))
    System.out.println("str1 == str2");
else
    System.out.println("str1 != str2");
if(str1.equals(str3))
    System.out.println("str1 == str3");
else
    System.out.println("str1 != str3");
result = str1.compareTo(str3);
if(result == 0)
    System.out.println("str1 = str3 ");
else
if(result < 0)
    System.out.println("str1 < str3");
else
    System.out.println("str1 > str3");
```

anhminhdo81@gmail.com

CHUỖI

```
        // Tao chuoi moi cho str4
        String str4 = "Mot Hai Ba Mot";
        idx = str4.indexOf("Mot");
        System.out.println("str4:" + str4);
        System.out.println(
                "Vi tri xuat hien dau tien cua chuoi con 'Mot' trong str4: " +
    idx);
        idx = str4.lastIndexOf("Mot");
        System.out.println("Vi tri xuat hien sau cung cua
        chuoi con 'Mot' trong str4:" + idx);
    }
}
```

CHUỖI

- **Chương trình nhập vào một chuỗi và in ra chuỗi nghịch đảo của chuỗi nhập.**

```
import java.lang.String;
import java.io.*;
public class InverstString
{    public static void main(String arg[])
     {   System.out.println("\n *** CHUONG TRINH IN CHUOI
                                                    NGUOC *** ");
         try
         {  System.out.println("\n *** Nhap chuoi:");
            BufferedReader in = new BufferedReader(new
                                    InputStreamReader(System.in));
         // Class BufferedReader cho phép đọc text từ luồng nhập ký
         //tự, tạo bộ đệm cho những ký tự để hỗ trợ cho việc đọc
         //những ký tự, những mảng hay những dòng.
```

Khoa Công Nghệ Thông Tin

CHUỖI

```
        // Doc 1 dong tu BufferReadered ket thuc bang dau ket thuc dong.
        String str = in.readLine();
        System.out.println("\n Chuoi vua nhap la:" + str);
        // Xuat chuoi nghich dao
        System.out.println("\n Chuoi nghich dao la:");
        for (int i=str.length()-1; i>=0; i--)
        {
                System.out.print(str.charAt(i));
        }
    }
    catch (IOException e)
    {
        System.out.println(e.toString());
    }
  }
}
```

CHUỖI

- **Lấy chuỗi con của một chuỗi**

```
class SubStr
{
        public static void main(String args[])
        {
                String orgstr = "Mot Hai Ba Bon";
                // Lay chuoi con dung ham
                // public String substring(int beginIndex, int
                // endIndex)
                String substr = orgstr.substring(4, 7);
                System.out.println("Chuoi goc: " + orgstr);
                System.out.println("Chuoi con: " + substr);
        }
}
```

CHUỖI

- **Mảng các chuỗi**

```
class StringArray
{    public static void main(String args[])
     {   String str[] = {"Mot", "Hai", "Ba", "Bon" };
         System.out.print("Mang goc: ");
         for(int i=0; i < str.length; i++)
                  System.out.print(str[i] + " ");
                  System.out.println("\n");
         // Thay doi chuoi
         str[0] = "Bon"; str[1] = "Ba";
         str[2] = "Hai"; str[3] = "Mot";
         System.out.print("Mang thay doi:");
         for(int i=0; i < str.length; i++)
                  System.out.print(str[i] + " ");
         System.out.print("\n");
     }
}
```


anhminhde81@gmail.com

Phương thức (Methods in Classes)

- Phương thức được định nghĩa như là một hành động hoặc một tác vụ thật sự của đối tượng
- Cú pháp

```
access_specifier modifier datatype
    method_name(parameter_list)
{
//body of method
}
```

Ví dụ về sử dụng phương thức

```
class Ex3 {
        int x = 10;                    // variable
    public void show( ) {    // method
        System.out.println(x);
    }
    public static void main(String args[ ]) {
       Ex3 t = new Ex3( );             // tao doi tuong t
        t.show( );                          // method call
       Ex3 t1 = new Ex3( );         // tao doi tuong t1
        t1.x = 20;
        t1.show( );
    }
}
```

Các chỉ định truy xuất của phương thức (Access specifiers)

- Công cộng (public)
- Riêng tư (private)
- Bảo vệ (protected)

Các toán tử

- Các loại toán tử:
  - Toán tử số học (Arithmetic operators)
  - Toán tử dạng Bit (Bitwise operators)
  - Toán tử so sánh (Relational operators)
  - Toán tử logic (Logical operators)
  - Toán tử điều kiện (Conditional operator)
  - Toán tử gán (Assignment operator)

anhminhde81@gmail.com

Toán tử số học
Arithmetic Operators

+	Addition (Phép cộng)
-	Subtraction (Phép trừ)
*	Multiplication (Phép nhân)
/	Division (Phép chia)
%	Modulus (Lấy số dư)
++	Increment (Tăng dần)
--	Decrement (Giảm dần)

anhminhde81@gmail.com



Khoa Công Nghệ Thông Tin

+=	Phép cộng và gán
-=	Phép trừ và gán
*=	Phép nhân và gán
/=	Phép chia và gán
%=	Phép lấy số dư và gán

anhminhde81@gmail.com

Toán tử Bit (Bitwise Operators)

~	Phủ định (NOT)
&	Và (AND)
|	Hoặc (OR)
^	Exclusive OR (XOR)
>>	Dịch sang phải (Shift right)
<<	Dịch sang trái (Shift left)

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

Toán tử so sánh (Relational Operators)

==	So sánh bằng
!=	So sánh khác
<	Nhỏ hơn
>	Lớn hơn
<=	Nhỏ hơn hoặc bằng
>=	Lớn hơn hoặc bằng

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

Toán tử Logic (Logical Operators)

&&	Logical AND
||	Logical OR
!	Logical unary NOT

Khoa Công Nghệ Thông Tin

anhminhde81@gmail.com

Toán tử điều kiện (Conditional Operator)

- Cú pháp

  Biểu thức 1 ? Biểu thức 2 : Biểu thức 3;

- **Biểu thức 1**

  Điều kiện kiểu Boolean trả về giá trị True hoặc False
- **Biểu thức 2**

  Trả về giá trị nếu kết quả của mệnh đề 1 là True
- **Biểu thức 3**

  Trả về giá trị nếu kết quả của mệnh đề 1 là False

Toán tử gán (Assignment Operator)

= Assignment (Phép gán)

Giá trị có thể được gán cho nhiều biến số

- Ví dụ

  a = b = c = d = 90;

anhminhde81@gmail.com

Thứ tự ưu tiên của các toán tử

Thứ tự	Toán tử
1.	trong ngoặc tính trước
2.	Các toán tử đơn như +,-,++,--
3.	Các toán tử số học và các toán tử dịch như *,/,+,-,<<,>>
4.	Các toán tử quan hệ như >,<,>=,<=,= =,!=
5.	Các toán tử logic và Bit như &&,II,&,I,^
6.	Các toán tử gán như =,*=,/=,+=,-=

- Thứ tự của các toán tử có thể được thay đổi bằng cách sử dụng các dấu ngoặc đơn trong mệnh đề

Các kí tự định dạng xuất dữ liệu (Escape Sequences)

Escape Sequence	**Mô tả**
\n	Xuống dòng mới
\r	Chuyển con trỏ đến đầu dòng hiện hành
\t	Chuyển con trỏ đến vị trí dừng Tab kế tiếp (ký tự Tab)
\\	In dấu \
\’	In dấu nháy đơn (’)
\”	In dấu nháy kép (”)

Các lệnh điều khiển

- Điều khiển rẻ nhánh:
  - Mệnh đề **if-else**
  - Mệnh đề **switch-case**
- Vòng lặp (Loops):
  - Vòng lặp **while**
  - Vòng lặp **do-while**
  - Vòng lặp **for**



HONGBANG
INTERNATIONAL
UNIVERSITY

❖Gói & Interface
❖(Packages & Interfaces)

Giới thiệu

- **Những thành phần cơ bản của 1 chương trình Java:**
  - Gói (Packages)
  - Giao diện (Interfaces)
- **Những phần của một chương trình Java:**
  - Lệnh khai báo gói (**package)**
  - Lệnh chỉ định gói được dùng (Lệnh i**mport)**
  - Khai báo lớp public (một file java chỉ chứa 1 lớp public class)
  - Các lớp khác (classes private to the package)
- Tập tin nguồn Java có thể chứa tất cả hoặc một vài trong số các phần trên.

anhminhde81@gmail.com

Interfaces

- Interface chính là lớp abstract thuần tuý
- Khai báo những phương thức mà lớp sử dụng
- Trong Java 1 lớp chỉ có thể dẫn xuất từ 1 lớp duy nhất tại cùng một thời điểm, nhưng có thể dẫn xuất cùng lúc nhiều Interface
- Trong Interface không chứa những phương thức cụ thể (concrete methods)
- interface cần phải được implements.

anhminhde81@gmail.com

Các bước tạo interface

- Định nghĩa Interface
- Biên dịch Interface
- Implements Interface

- Tính chất của interface:
  - Tất cả phương thức trong interface phải là **public.**
  - Các phương thức phải được định nghĩa trong lớp dẫn xuất interface đó.

Sử dụng Interface

- Interface không thể dẫn xuất từ lớp khác, nhưng có thể dẫn xuất từ những interface khác
- Nếu một lớp dẫn xuất từ một interface mà interface đó dẫn xuất từ các interface khác thì lớp đó phải định nghĩa tất cả các phương thức có trong các interface đó.
  - Nếu không lớp đó trở thành lớp abstract
- Khi định nghĩa một interface mới có nghĩa là một kiểu dữ liệu tham chiếu mới được tạo ra.

Gói (Packages)

- Gói được xem tương tự như thư mục lưu trữ những lớp, những interfaces và các gói con khác.
- Nhóm các lớp (classes)
- giao diện (interfaces)
- Và gói con (sub-packages) có liên quan lại với nhau

anhminhde81@gmail.com

Gói (Packages)

- Những ưu điểm khi dùng gói (Package):
  - Cho phép tổ chức các lớp vào những đơn vị nhỏ hơn
  - Giúp tránh được tình trạng trùng lặp khi đặt tên lớp, tên interfaces .
  - Cho phép bảo vệ các lớp.
  - Tên gói (Package) có thể được dùng để nhận dạng chức năng của các lớp.

anhminhde81@gmail.com

Gói (Packages)

- Những lưu ý khi tạo gói:
  - Mã nguồn phải bắt đầu bằng lệnh ‘package’
  - Mã nguồn phải nằm trong cùng thư mục mang tên của gói
  - Tên gói nên bắt đầu bằng ký tự thường (lower case) để phân biệt giữa tên lớp đối tượng và tên gói
  - Những lệnh khác phải viết phía dưới dòng khai báo gói là mệnh đề **import**, kế đến là các lệnh định nghĩa lớp đối tượng
  - Những lớp đối tượng trong gói cần phải được biên dịch.
  - Để chương trình Java có thể sử dụng những gói này, ta phải **import** gói vào trong mã nguồn

Gói (Packages)

- Import gói (Importing packages):
- Có 2 cách:
  - Xác định lớp hoặc interface của gói cần được import.
    - Hoặc có thể import toàn bộ gói

    vd : import java.util.Vector;

    import java.util.*;

Các bước tạo ra gói (Package)

- Khai báo gói
- Import những gói chuẩn cần thiết
- Khai báo và định nghĩa các lớp đối tượng có trong gói
- Lưu thành tập tin .**java**, và biên dịch những lớp đối tượng đã được định nghĩa trong gói.

Sử dụng những gói do người dùng định nghĩa (user-defined packages)

- Mã nguồn của những chương trình này phải ở cùng thư mục của gói do người dùng định nghĩa.

→ Nếu không ta phải thiết lập đường dẫn.

- Để những chương trình Java khác sử dụng những gói này, import gói vào trong mã nguồn
- Import những lớp đối tượng cần dùng
- Import toàn bộ gói
- Tạo tham chiếu đến những thành viên của gói

Thiết lập đường dẫn - xác lập CLASSPATH

- là một **khái niệm/biến môi trường** của Java, dùng để chỉ đường dẫn dẫn tới nơi chứa các file đã biên dịch (.class) hoặc các file thư viện nén (.jar, .zip).
- Là danh sách các thư mục, giúp cho việc tìm kiếm các tập tin lớp đối tượng tương ứng
- Nên xác lập CLASSPATH trong lúc thực thi (runtime), vì như vậy nó sẽ xác lập đường dẫn cho quá trình thực thi hiện hành

anhminhde81@gmail.com

Thiết lập đường dẫn - xác lập CLASSPATH

- Dùng cờ -classpath (hoặc -cp) khi chạy câu lệnh Terminal
- Đây là cách phổ biến và khuyến nghị nhất khi làm việc trực tiếp bằng câu lệnh:
  - javac -cp "lib/DSTT.jar" Main.java
  - java -cp ".;lib/DSTT.jar" Main
- Thường Classpath tự động:
  - Các IDE như IntelliJ IDEA, Eclipse, … tự động cấu hình Classpath dựa vào thư mục đầu ra (out/ hoặc bin/).
  - Các công cụ Maven (pom.xml) hay Gradle (build.gradle) sẽ tự động tải các thư viện file .jar về và nạp chính xác vào Classpath khi biên dịch/chạy dự án.

Gói và điều khiển truy xuất (Packages & Access Control)

	public	protected	No modifier	private
Same class	Yes	Yes	Yes	Yes
Same package subclass	Yes	Yes	Yes	No
Same package non-subclass	Yes	Yes	Yes	No
Different package subclass	Yes	Yes	No	No
Different package non-subclass	Yes	No	No	No

anhminhde81@gmail.com

Gói java.lang

- Gói **java.lang** được import mặc định.
- Những lớp Wrapper cho các kiểu dữ liệu nguyên thủy:

Data type	Wrapper class
boolean	Boolean
byte	Byte
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

Gói java.lang

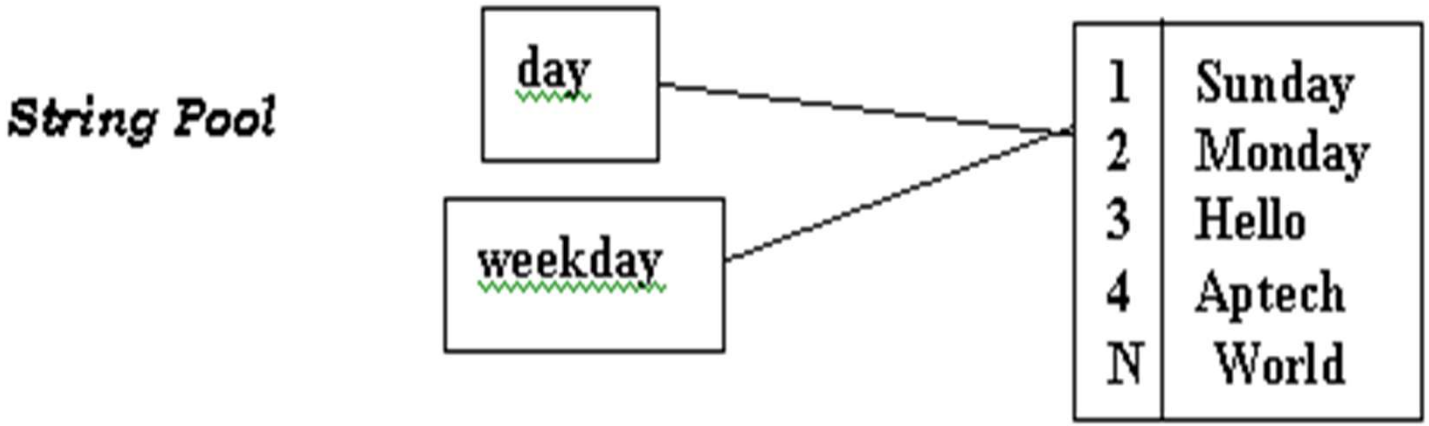
- Các lớp phổ biến:
  - Object:
  - Class:
  - String, StringBuilder, StringBuffer
  - Math:
  - System:
  - Runtime:
  - Thread:
  - ClassLoader:
  - Process, ProcessBuilder:

Lớp String

- Phương thức khởi tạo (Constructor):
  - **String str1 = new String();**
  - **String str2 = new String("Hello World");**
  - **char ch[ ] = {"A","B","C","D","E"};**
  - **String str3 = new String(ch);**
  - **String str4 = new String(ch,0,2);**

String Pool

- 'String Pool' đại diện cho tất cả các ký tự được tạo ra trong chương trình
- Khái niệm 'String Pool'



Những phương thức của lớp **String**

- **charAt( )//**
- **startsWith()**
- **endsWith( )**
- **copyValueOf( )**
- **toCharArray( )**
- **indexOf( )**
- **toUpperCase( )**
- **toLowerCase( )**
- **trim( )**
- **equals( )**

Java SE Documentation (Javadoc)	Kho tài liệu chính thức chi tiết nhất về các lớp (class), giao diện (interface) và phương thức (method) trong Java API.
The Java Tutorials (Oracle)	Hệ thống bài giảng, hướng dẫn từ cơ bản đến nâng cao do Oracle biên soạn cho lập trình viên.
Oracle DevLive & Stack Overflow	Diễn đàn thảo luận chính thức của Oracle kết hợp với cộng đồng **Stack Overflow** (nơi tập trung phần lớn thảo luận về Java).

Lớp StringBuffer

- Cung cấp những phương thức khác nhau để thao tác trên đối tượng string (chuỗi ký tự)
- Những đối tượng của lớp này khá linh hoạt
- Cung cấp những phương thức khởi tạo (constructor) đã được nạp chồng (overloaded)
- Những phương thức của lớp **StringBuffer**:
  - **append( )** // noi chuoi
  - **insert( )** //chen
  - **charAt( )// ktra ky tu ko?**
  - **setCharAt( ) //thietlapktu**
  - **setLength( )**
  - **getChars( )**
  - **reverse( ) //dao chuoi**

anhminhde81@gmail.com

Lớp **java.lang.Math**
→ 5 nhóm hỗ trợ toán học

- **abs()**
- **ceil()// lam tron tren**
- **floor()// lam tron duoi**
- **max()**
- **min()**
- **round()**
- **random()**
- **sqrt()**
- **sin()**
- **cos()**
- **tan()**

Lớp Runtime

- Trong Java.lang và 4 chức năng
- Mỗi ứng dụng Java khi khởi chạy chỉ có duy nhất **một instance** (đối tượng) của lớp Runtime -
- Đóng gói (Encapsulates) môi trường thực thi
- Dùng để quản lý bộ nhớ, và thi hành những tiến trình cộng thêm
- Phương thức:
  - **exit(int)**
  - **freeMemory( )**
  - **getRuntime( )**
  - **gc( )**
  - **totalMemory( )**
  - **exec(String)**

Lớp System

- Đánh dấu final và khởi tạo là private → không khởi tạo được và không kế thừa được
- Cung cấp những luồng chuẩn như nhập (Input), xuất (Output) và các luồng lỗi (Error Streams)
- Cung cấp khả năng truy xuất đến những thuộc tính của hệ thống thực thi Java, và những thuộc tính môi trường như phiên bản, đường dẫn, nhà cung cấp…
- Phương thức:
  - **exit(int)**
  - **gc()**
  - **getProperties()**
  - **setProperties()**
  - **currentTimeMillis()**
  - **arraCopy(Object, int, Object, int, int)**

Lớp Class

- Trong java.lang →đại diện cho bản thiết kế các: lớp, interface, enum, mảng hay kiểu dữ liệu nguyên thủy → tại thời điểm chương trình đang chạy - **runtime**
- Đối tượng của lớp này che giấu trạng thái thực thi của đối tượng trong một ứng dụng Java
- Đối tượng của lớp này có thể tạo ra bằng 1 trong 3 cách sau:
  - Sử dụng phương thức **getClass( )** của đối tượng
  - Sử dụng phương thức tĩnh **forName( )** của lớp để tạo ra một đối tượng của lớp đó trong lúc đặt tên cho lớp
  - Sử dụng đối tượng ClassLoader để nạp một lớp mới

anhminhde81@gmail.com

Lớp Object

- Là lớp cha (gốc) tối cao của tất cả các lớp
- Phương thức:
  - **equals(Object)**
  - **finalize()**
  - **notify()**
  - **notifyAll()**
  - **toString()**
  - **wait()**
  - **clone()**

Gói java.util

- Cung cấp phần lớn những lớp Java hữu dụng và thường xuyên (tiện tích) cần đến trong hầu hết các ứng dụng
- Giới thiệu những lớp trừu tượng sau:
  - Hashtable
  - Random
  - Vector
  - StringTokenizer

Lớp Hashtable

- Dẫn xuất từ lớp trừu tượng Dictionary
- Dùng để nối kết những khóa vào những giá trị cụ thể
- Phương thức khởi tạo Hashtable:
  - **Hashtable(int)**
  - **Hashtable(int, float)**
  - **Hashtable( )**

Những phương thức của lớp **Hashtable**

- **clear()**
- **done()**
- **contains(Object)**
- **containsKey(Object)**
- **elements()**
- **get(Object key)**
- **isEmpty()**
- **keys()**
- **put(Object, Object)**
- **rehash()**
- **remove(Object key)**
- **size()**
- **toString()**

Lớp Random

- 3 cách: lớp **java.util.Random**; hàm **Math.random()**; **ThreadLocalRandom** (tối ưu nhất cho đa luồng).
- Những phương thức nhận giá trị ngẫu nhiên:
  - **nextDouble( )**
  - **nextFloat( )**
  - **nextGaussian( )**
  - **nextInt( )**
  - **nextLong( )**
- Phương thức khởi tạo (Constructors):
  - **random()**
  - **random(long)**

Những phương thức của lớp Random

- nextDouble()
- nextFloat()
- nextGaussian()
- nextInt()
- nextLong()
- setSeed(long)

Lớp Vector

- Tất cả phương thức đều đánh dấu synchronized
- Triển khai interface List
- Cung cấp khả năng co giản cho mảng khi thêm phần tử vào mảng
- Lưu trữ những thành phần của kiểu Object
- Một Vector riêng rẽ có thể lưu trữ những phần tử khác nhau, đó là những instance của những lớp khác nhau
- Phương thức khởi tạo (Constructors):
  - **Vector(int)**
  - **Vector(int, int)**
  - **Vector()**

anhminhde81@gmail.com

Những phương thức của lớp **Vector**

- **addElement(Object)**
- **capacity( )**
- **clone( )**
- **contains(Object)**
- **copyInto(Object [ ])**
- **elementAt(int)**
- **elements( )**
- **ensureCapacity(int)**
- **firstElement( )**
- **indexOf(Object)**
- **indexOf(Object, int)**
- **insertElementAt(Object, int)**
- **isEmpty( )**
- **lastElement( )**
- **lastIndexOf(Object)**
- **lastIndexOf(Object, int)**
- **removeAllElements( )**
- **removeElement(Object)**
- **removeElementAt(int)**
- **setElementAt(Object, int)**
- **setSize(int)**
- **size( )**
- **toString( )**
- **trimToSize( )**

Lớp StringTokenizer

- Trong java.ulti
- Có thể được dùng để tách một chuỗi thành những thành phần cấu thành của nó (constituent tokens)
- Ký tự phân cách có thể được chỉ định khi một đối tượng **StringTokenizer** được khởi tạo
- Phương thức khởi tạo (Constructors):
  - **StringTokenizer(String)**
  - **StringTokenizer(String, String)**
  - **StringTokenizer(String, String, Boolean)**
- Lớp **StringTokenizer** sử dụng giao diện liệt kê (enumeration interface)

anhminhde81@gmail.com

Những phương thức của lớp StringTokenizer

- countTokens( )
- hasMoreElements( )
- hasMoreTokens( )
- nextElement( )
- nextToken( )
- nextToken(String)